

Chapter 6. CMake build configuration

Use CMake cache variables to customize SDK builds.

6.1. Full List of SDK Configuration Variables

Table 35. SDK CMake Configuration Variables

Name / Description	Default
PICO_BARE_METAL Flag to exclude anything except base headers from the build	0
PICO_BOARD Board name being built for. This may be specified in the user environment (see Section 6.2)	pico or pico2
PICO_BOARD_CMAKE_DIRS List of directories to look for <PICO_BOARD>.cmake in. This may be specified in the user environment	
PICO_BOARD_HEADER_DIRS List of directories to look for <PICO_BOARD>.h in. This may be specified the user environment	
PICO_CMAKE_PRELOAD_PLATFORM_FILE Custom CMake file to use to set up the platform environment	
PICO_COMPILER Specifies the compiler family to use (see Section 6.3)	PICO_DEFAULT_COMPILER which is set based on PICO_PLATFORM
PICO_CONFIG_HEADER_FILES List of extra header files to include from pico/config.h for all platforms	
PICO_COPY_TO_RAM Option to default all binaries to copy code from flash to SRAM before running (see Section 6.4)	0
PICO_CXX_ENABLE_CXA_ATEXIT Enable cxa-atexit	0
PICO_CXX_ENABLE_EXCEPTIONS Enable CXX exception handling	0
PICO_CXX_ENABLE_RTTI Enable CXX rtti	0
PICO_DEFAULT_BOOT_STAGE2 Simpler alternative to specifying PICO_DEFAULT_BOOT_STAGE2_FILE where the latter is set to src/rp2_common/boot_stage2/{PICO_DEFAULT_BOOT_STAGE2}.S	compile_time_choice
PICO_DEFAULT_BOOT_STAGE2_FILE Default boot stage 2 file to use unless overridden by pico_set_boot_stage2 on the TARGET; this setting is useful when explicitly setting the default build from a per board CMake file	

Name / Description	Default
PICO_DEFAULT_PIOASM_OUTPUT_FORMAT Default output format used by pioasm when using pico_generate_pio_header	c-sdk
PICO_DEFAULT_RP2350_PLATFORM RP2350 Default actual platform to build for if rp2350 is specified for PICO_PLATFORM e.g. rp2350-arm-s/rp2350-riscv	rp2350-arm-s
PICO_DEFAULT_UART_BAUD_RATE Define the default UART baudrate	115200
PICO_GCC_TRIPLE List of GCC_TRIPLES – usually only one – to try when searching for a compiler. This may be specified the user environment (see Section 6.3)	PICO_DEFAULT_GCC_TRIPLE which is set based on PICO_COMPILER
PICO_HOST_CONFIG_HEADER_FILES List of extra header files to include from pico/config.h for the host platform only	
PICO_NO_COPRO_DIS Disable disassembly listing postprocessing that disassembles RP2350 coprocessor instructions	0
PICO_NO_FLASH Option to default all binaries to not use flash i.e. run from SRAM (see Section 6.4)	0
PICO_NO_GC_SECTIONS Disable <code>-ffunction-sections -fdata-sections</code> and <code>--gc-sections</code>	0
PICO_NO_HARDWARE Option as to whether the build is not targeting an RP2040 or RP2350 device	1 when PICO_PLATFORM is host, 0 otherwise
PICO_NO_PICOTOOL Disable use/requirement for picotool meaning that UF2 output and signing/hashing and coprocessor disassembly will all be unavailable	0
PICO_NO_TARGET_NAME Don't define PICO_TARGET_NAME	0
PICO_NO_UF2 Disable UF2 output	0
PICO_ON_DEVICE Option as to whether the build is targeting an RP2040 or RP2350 device	0 when PICO_PLATFORM is host, 1 otherwise
PICO_PLATFORM Platform to build for e.g. rp2040/rp2350/rp2350-arm-s/rp2350-riscv/host. This may be specified in the user environment (see Section 6.2)	based on PICO_BOARD or environment value
PICO_RP2040_CONFIG_HEADER_FILES RP2040 List of extra header files to include from pico/config.h for the rp2040 platform only	
PICO_RP2350_ARM_S_CONFIG_HEADER_FILES RP2350 List of extra header files to include from pico/config.h for the rp2350-arm-s platform only	
PICO_RP2350_RISCV_CONFIG_HEADER_FILES RP2350 List of extra header files to include from pico/config.h for the riscv platform only	

Name / Description	Default
PICO_SDK_VERSION_MAJOR SDK major version number	Current SDK major version
PICO_SDK_VERSION_MINOR SDK minor version number	Current SDK minor version
PICO_SDK_VERSION_PRE_RELEASE_ID Optional SDK pre-release version identifier	Current SDK pre-release identifier
PICO_SDK_VERSION_REVISION SDK version revision	Current SDK revision
PICO_SDK_VERSION_STRING SDK version string	Current SDK version string
PICO_STDIO_RTT Option to globally enable stdio RTT for all targets by default	0
PICO_STDIO_SEMIHOSTING Option to globally enable stdio semi-hosting for all targets by default	0
PICO_STDIO_UART Option to globally enable stdio UART for all targets by default	1
PICO_STDIO_USB Option to globally enable stdio USB for all targets by default	0
PICO_STDIO_USB_CONNECT_WAIT_TIMEOUT_MS Maximum number of milliseconds to wait during initialization for a CDC connection from the host (negative means indefinite) during initialization	0
PICO_TOOLCHAIN_PATH Path to search for compiler (see Section 6.3)	none (i.e. search system paths)
PICO_USE_DEFAULT_MAX_PAGE_SIZE Don't shrink linker max page to 4096	0

6.2. Platform and Board Configuration

Passing `PICO_BOARD=my_board_name` to the CMake build (or specifying it in your environment) will cause the header `my_board_name.h` to be included by all other SDK headers in order to provide `#defines` particular to the board you are using.

You may also wish to specify your own board configuration in which case you can set `PICO_BOARD_HEADER_DIRS` in the environment or CMake to a semicolon separated list of paths to search for `my_board_name.h`.

On previous versions of the SDK there was generally little need for setting `PICO_PLATFORM` as the default value `rp2040` selected RP2040 - the one and only RP-series microcontroller at the time.

SDK version 2.0.0 now supports the following RP-series microcontroller platforms along with the pre-existing value `host` that can be used to build code for testing.

`rp2040`

Building for RP2040

`rp2350-arm-s`

Building for RP2350 on Arm processors; the "s" stands for secure, and means the binary runs directly from the bootrom, when the processor is still in secure mode.

`rp2350-riscv`

Building for RP2350 on RISC-V processors.

Individual manufactured boards will usually support only one of either RP2040 or RP2350. To avoid having to specify `PICO_PLATFORM` in addition to `PICO_BOARD`, specifying the latter can now automatically set the former (or vice versa).

The following steps are applied in order, with the results from the previous step being used in the next:

1. If neither `PICO_BOARD` or `PICO_PLATFORM` is specified, `PICO_PLATFORM` defaults to `PICO_DEFAULT_PLATFORM` which itself defaults to `rp2040`
2. If `PICO_PLATFORM` is specified and not `PICO_BOARD`, then `PICO_BOARD` is defaulted based on the value of `PICO_PLATFORM`:
 - `pico` for `PICO_PLATFORM=rp2040`
 - `pico2` for `PICO_PLATFORM=rp2350-arm-s` or `PICO_PLATFORM=rp2350-riscv`
3. If `PICO_BOARD` is specified but not `PICO_PLATFORM`, `PICO_PLATFORM` will be set if a value for it is specified in the board header.

Because most RP2350 boards allow both Arm and RISC-V development, `rp2350` is also a valid value for `PICO_PLATFORM`, and is often specified by a board header in step 3 above, but is always replaced with the value of `PICO_DEFAULT_RP2350_PLATFORM` to allow the user their own preference. `PICO_DEFAULT_RP2350_PLATFORM` defaults to `rp2350-arm-s` if not otherwise specified.

i NOTE

Both `PICO_PLATFORM` and `PICO_BOARD` are **latched** if they have been specified via the environment, on the first CMake configuration; i.e. the value from the environment will not be used when configuring CMake subsequently in the same existing build directory.

6.3. Compiler and Toolchain Configuration

The SDK supports building for Arm Cortex-M0 plus processors on RP2040 and for both Arm Cortex-M33 processors and RISC-V Hazard3 processors on RP2350.

The SDK also supports building with either GCC or LLVM (clang) on Arm. See [Section 2.10](#) for more details of supported compilers.

6.3.1. Variables

The following variables are used to find and configure the right compiler.

6.3.1.1. PICO_COMPILER

This is usually defaulted for you to a GCC compiler based on `PICO_PLATFORM`. However, you can select one of the following values

- `pico_arm_gcc` - Selects `pico_arm_cortex_m0plus_gcc` on RP2040 and `pico_arm_cortex_m33_gcc` on RP2350
- `pico_arm_cortex_m0plus_gcc` - Configures GCC to build for Arm Cortex-M0 plus
- `pico_arm_cortex_m33_gcc` - Configures GCC to build for Arm Cortex-M33
- `pico_arm_clang` - Selects `pico_arm_cortex_m0plus_clang` on RP2040 and `pico_arm_cortex_m33_clang` on RP2350
- `pico_arm_cortex_m0plus_clang` - Configures LLVM/clang to build for Arm Cortex-M0 plus
- `pico_arm_cortex_m33_clang` - Configures LLVM/clang to build for Arm Cortex-M33
- `pico_riscv_gcc` - Configures GCC to build for RISC-V Hazard3
- `pico_riscv_gcc_zcb_zcmp` - Configures GCC to build for RISC-V Hazard3 using `zcb` and `zcmp` extensions that aren't supported by all compilers

6.3.1.2. PICO_GCC_TRIPLE

This specifies one or more compiler "triples" to try when looking for a GCC compiler.

On Arm this defaults to `arm-none-eabi`.

On RISC-V this defaults to `riscv32-unknown-elf;riscv32-corev-elf` i.e. the two most common options are supported.

6.3.1.3. PICO_TOOLCHAIN_PATH

Armed with `PICO_COMPILER` and `PICO_GCC_TRIPLE` (if using GCC) the SDK will then search for a compiler. By default, it searches the path, but `PICO_TOOLCHAIN_PATH` may be set to specify the root directory of a compiler toolchain install.

6.3.1.4. PICO_CLIB

Most programs for the SDK require a C-library. Generally your installed compiler will include the toolchain. In this case, the SDK will try to detect either of the following runtimes, as which one is used effects how the SDK interacts with it.

- `newlib`
- `picolibc`
- `llvm-libc`

The SDK sets `PICO_CLIB` to one of these values, however you can set it yourself first if you want to force a choice.

6.4. Binary Type configuration

These variables control how executables for RP-series microcontroller are laid out in memory. The default is for the code and data to be entirely stored in flash with writable data (and some specifically marked) methods to copy into RAM at startup.

Variable name	Values	Result
PICO_DEFAULT_BINARY_TYPE	<code>default</code>	Stores binaries in flash storage. Runs binaries from flash storage.
	<code>no_flash</code>	Stores binaries in memory. Runs binaries from memory. Does not require any flash storage. Note: You must reload this type of binary after every reboot via UF2 file or debugger.
	<code>copy_to_ram</code>	Stores binaries in flash, but copies them to memory (RAM) before executing.
	<code>blocked_ram</code>	
<code>PICO_NO_FLASH</code>	<code>0 / 1</code>	Equivalent to <code>PICO_DEFAULT_BINARY_TYPE=no_flash</code> if <code>=1</code> .
<code>PICO_COPY_TO_RAM</code>	<code>0 / 1</code>	Equivalent to <code>PICO_DEFAULT_BINARY_TYPE=copy_to_ram</code> if <code>=1</code> .
<code>PICO_USE_BLOCKED_RAM</code>	<code>0 / 1</code>	Equivalent to <code>PICO_DEFAULT_BINARY_TYPE=blocked_ram</code> if <code>=1</code> .

 TIP

You can set the binary type for each executable target (as created by `add_executable`) by calling `pico_set_binary_type(target type)` using the same type as `PICO_DEFAULT_BINARY_TYPE`.